European Journal of Operational Research 196 (2009) 877–885

Contents lists available at ScienceDirect

European Journal of Operational Research

journal homepage: www.elsevier.com/locate/ejor

Discrete Optimization

Timetabling problems at the TU Eindhoven

John van den Broek *, Cor Hurkens, Gerhard Woeginger

Department of Mathematics and Computer Science, Eindhoven University of Technology, P.O. Box 513, Eindhoven, The Netherlands

ARTICLE INFO

Article history:
Received 16 April 2007
Accepted 25 April 2008
Available online 13 May 2008

Keywords:
Integer programming
University timetabling
Network flow formulation
NP-completeness
Lexicographical optimization

ABSTRACT

The students of the Department of Industrial Design at the TU Eindhoven are allowed to design part of their curriculum by selecting courses from a huge course pool. They do this by handing in ordered preference lists with their favorite courses for the forthcoming time period. Based on this information (and on many other constraints), the department then assigns courses to students. Until recently, the assignment was computed by human schedulers who used a quite straightforward greedy approach. In 2005, however, the number of students increased substantially, and as a consequence the greedy approach did not yield acceptable results anymore.

This paper discusses the solution of this real-world timetabling problem. We present a complete mathematical formulation and explain all the constraints resulting from the situation in Eindhoven. We solve the problem using lexicographical optimization with four subproblems. For all four subproblems, an elegant integer linear programming model is given which easily can be put into CPLEX. Finally, we report on our computational experiments and results around the Eindhoven real-world data.

© 2008 Elsevier B.V. All rights reserved.

1. Introduction

In February 2005, outraged students of the Department of Industrial Design were protesting at the TU Eindhoven (The Netherlands). Why were they doing this? The academic year of these roughly 350 students of Industrial Design is split into a number of periods. In every period, every student must do a number of small *courses*. There is a pool of roughly 55 courses to choose from, and every student submits an ordered preference list with his/her 10 favorite courses to the department. Based on all the ordered preference lists, a *scheduler* of the department then assigns roughly 4 courses to every student.

The scheduler was a human decision-maker who essentially applied a hand-woven greedy assignment procedure, starting filling the most popular courses with students with the lowest id-numbers. In February 2005, the students were very dissatisfied with the work of the human scheduler. More than 150 out of the 350 students were assigned to fewer courses than requested. Therefore, they needed to do courses that were not on their preference list.

The people from the Department of Industrial Design realized that they had a problem and they did not know how to settle this problem themselves. The number of students had increased substantially, and the timetabling problem had become much larger, much harder, and much more complex. Hence, they turned to us, mathematicians. They explained their problem (in problem formulation No. 1), and we happily told them that we were able to solve it: the problem (in formulation No. 1) can be modeled as a network flow problem, and hence is solvable in polynomial time. Unfortunately, it turned out that formulation No. 1 was not a complete formulation of the problem. They had forgotten to inform us about a number of additional restrictions that led to a new, more difficult assignment problem (in formulation No. 2), which eventually turned out to be NP-hard.

This paper is a report on the course assignment problem of the Department of Industrial Design. We will describe the assignment problem in its (incomplete) formulation No. 1 and in its (complete) formulation No. 2. We show that formulation No. 1 yields a tractable problem, whereas formulation No. 2 does not. Our main contribution is a careful case study of the complete problem formulation. We apply lexicographical optimization and design four elegant integer linear programming models for it, with roughly 9000 variables and roughly 7000 constraints. Putting these ILP models into CPLEX yields excellent results within moderate computation times. We describe the ILP models in detail, and we report on our computational experiments with the real-world data of the Department of Industrial Design.

Structure of the paper. The rest of the paper is structured in the following way. In Section 2 we give a literature review of university and school time tabling. Section 3 contains a detailed description of the problem we solved for the Department of Industrial Design. We prove that the timetabling problem in formulation No. 2 is NP-hard, by proving some independent subproblems to be NP-hard. These proofs are given in Section 4. The problem is formulated as

* Corresponding author. Tel.: +31 402472593; fax: +31 402465995.
E-mail address: j.j.j.v.d.broek@tue.nl (J. van den Broek).

0377-2217/$ - see front matter © 2008 Elsevier B.V. All rights reserved.
doi:10.1016/j.ejor.2008.04.038

an integer linear program, which will be described in Section 5. Section 6 contains the computational results and conclusions are given in Section 7.

2. Literature review

The literature contains a large number of variants of the timetabling problem. These variants differ from each other by the type of institution involved (university or high school) and by the type of constraints. The annotated bibliography of timetable construction by Schmidt and Ströhlein (1980) lists many papers that appeared before 1980. Schaerf (1999) gives a survey of the various formulations of timetabling problems and classifies the timetabling problem into the following three main classes: school timetabling, examination timetabling and course timetabling. Of course this classification is crude, and there are many real-world timetabling problems that fall in between two of these classes. Surveys of timetabling methods and applications are de Werra (1985), Burke et al. (1997), Carter and Laporte (1998) and Burke and Petrovic (2002).

The basic school timetabling problem is also known as the *class-teacher model*. A *class* is a group of students taking an identical set of courses and typically remaining together throughout the week. The simplest problem consists in assigning lectures to periods in such a way that no teacher or class is involved in more than one lecture at a time. Even et al. (1976) proved that there always exists a solution of this simplest version of the school timetabling problem, unless a teacher or class is involved in more lectures than there are time slots. Alternative formulations of the school timetabling problem with more constraints can be found for example in Even et al. (1976), Garey and Johnson (1979) and de Werra (1985). Daskalaki and Birbas (2005) provide an integer programming formulation of the class-teacher problem and solve it with a two-stage relaxation procedure. The problem is formulated as a set packing problem with side constraints by Avella and Vasilev (2005). They tighten the formulation by adding the valid inequalities of the Set Packing polytope and introduce some new valid inequalities.

University timetabling can be classified into two categories: course and examination timetabling. Petrovic and Burke (2004) discuss problem statements and give an overview on recent research results on university timetabling. The main differences between course timetabling and examination timetabling are that examination timetabling has only one exam for each course, that the time conflict condition is strict, and that several exams can be done simultaneously in one room. Examples for additional soft constraints are: Students can do at most one exam per day, and students may not have too many consecutive exams. Schaerf (1999) gives an integer linear programming formulation of the examination timetabling problem and describes some alternative variants of the problem. Carter and Laporte (1995) provide an overview on examination timetabling.

The course timetabling problem consists in scheduling a set of lectures for each course within a given number of rooms and time period. A well-known course timetabling problem was used for the International Timetabling Competition in 2002 and is for example well described by Lewis and Paechter (2007). The main difference from the school timetabling problem is that university courses can have common students, whereas school classes are disjoint sets of students. de Werra (1985) gives a binary integer programming formulation of the basic course timetabling problem. An overview on course timetabling problems is given by Carter and Laporte (1998) and Schaerf (1999) discusses some of the most common variants of the basic formulation. The design and implementation of a decision support system for constructing a combined university course–examination timetable is reported by Dimopoulou and Miliotis (2001). They also take into account the increased flexibility of students' preferences for specific classes.

If the expected number of students for a course is too large for one room, then courses are split into multiple *sections* that can be taught by different teachers. In the *student scheduling problem (SSP)*, the weekly timetable is fixed and a section may take more than one time slot. Students are required to take a certain number of courses, which they have to select themselves after the timetable is made available. The problem consists of assigning a student to a specific section of a course for a given fixed timetable such that students are satisfied, there are no time conflicts, section sizes are balanced and room capacities are respected. Problem SSP clearly differs from the well-known course timetabling problem shortly described above. The main difference between the SSP and our timetabling problem is that for the SSP *all* courses on the preference list of the students have to be assigned to students. This results in most practical cases into an empty feasible solution set. Other differences are that we consider an ordered preference list and in the SSP case the maximum number of students for a course is a soft constraint.

Cheng et al. (2002) discuss problem SSP as it generally applies to high schools in North America. They define the problem as the assignment of students to sections of courses offered at various times during the week. The objective is to fulfil student requests, providing a conflict-free schedule. They show that the problem is NP-hard and discuss various multi-commodity flow formulations with fractional capacities and integral gains.

Laporte and Desrochers (1986) give a mathematical formulation of the student scheduling problem. They formulate the problem as an optimization problem splitting the requirements into hard and soft ones. The only hard constraint in their model is that student course selections must be respected. Time conflicts for students are soft constraints. When time conflicts occur students are advised to make a different course selection. The problem is then solved in three phases: In the first one the algorithm searches for an admissible solution, in the second section enrollments are balanced and in the third the room capacities have to be respected.

Tripathy (1992) formulated the student scheduling problem as an integer linear programming problem and uses Lagrangian Relaxation to solve it. Sabin and Winter (1986) use a greedy approach that is moderated by an intelligent ordering of the students. Miyaji et al. (1981) apply goal programming. The student scheduling problem is also considered in for example (Busam, 1967; Feldman and Golumbic, 1989).

3. Problem description

At our first meeting, the planner from the Department of Industrial Design explained the problem to us in a certain problem formulation No. 1; see Section 3.1. This problem can be modeled as a network flow problem, and hence is solvable in polynomial time; see Ahuja et al. (1993). This network flow model will also be given in Section 3.1.

Unfortunately, we learnt after some time that formulation No. 1 was *not* a complete formulation of the problem. The planner actually had forgotten to tell us about a number of additional restrictions that lead us to a new, more difficult assignment problem formulation No. 2. Section 3.2 describes formulation No. 2.

3.1. Problem formulation No. 1

At the first meeting with the Department of Industrial Design, we were told that every student hands in a preference list of at most 10 courses and requests a certain number of courses. The

only constraints are that a student cannot do two courses at the same time and there is a maximum number of students that can be assigned to a course. This subsection contains a more detailed description of problem formulation No. 1 and its network flow model.

All courses are provided in the set C of courses. For each course c an *upper bound* $C_c^{\max}$ on the number of students is given. This maximum number of students depends on the preference of the teacher and the room capacity in which the course is given. Each course has one weekly meeting time, which is already fixed. This weekly meeting time always consists of two consecutive hours. Two such consecutive hours are defined as one *time slot*. The weekly meeting time of a course is chosen from a set T of disjoint time slots. $T(c)$ is defined as the time slot that is the weekly meeting time of course c. Hence, one of the constraints in the model is that courses c_i and c_j cannot be assigned to one student if $T(c_i) = T(c_j)$.

We define S as the set of students. For each student s the requested number r_s of courses is given, which on average is about four. P_s is defined as the set of positions on the preference list for which student s filled in a course. Table 1 gives a few examples of preference lists. Column i gives the course on position i of the preference list of the corresponding student. The parameter c_{sp} is introduced as the course on position p of the preference list of student s.

In summary: the input of problem formulation No. 1 consists of:

- A set T of time slots.
- A set C of courses; for every course $c \in C$ a time slot $T(c)$ and a maximum number $C_c^{\max}$ of participating students is given.
- A set S of students; for every student $s \in S$ a set P_s of filled positions of the preference list, a course c_{sp} for each position $p \in P_s$ and a requested number r_s of courses is given.

The goal is to assign as many courses to students as possible, while:

- The number of courses assigned to student s does not exceed the requested number r_s.
- Courses assigned to a student are on his preference list.
- Courses assigned to a student do not conflict in time.
- No course exceeds its maximum number of assigned students.

As stated before, problem formulation No. 1 can be modeled as a network flow problem. An illustration of the network with its arc capacities can be found in Fig. 1. The layered network has a source node connected to the first layer of nodes of which each node corresponds to a student. The set of student nodes is denoted with V_S. The capacity of the arc between a student node and the source node is equal to the requested number of courses of the student.

For each student and each time slot, a node is defined in the second layer, which is denoted as the set of nodes V_T. Therefore, this set V_T contains $|S||T|$ nodes. Each student node has $|T|$ outgoing arcs with capacity one to a node in V_T that corresponds to a time slot.

The third layer of the graph contains a node for every course. This set of course nodes is defined as V_C. Each course is given in exactly one time slot. Therefore, if a course is on the preference list of the student, this course node is connected with the node in V_T that corresponds to its time slot and this student. For example, in Fig. 1 course number 1 is given in time slot 1. Because course 1 is chosen by student 1 and student $|S|$, there are arcs from nodes t_{11} and $t_{|S|1}$ to course node C_1.

Each course node is connected to the sink with its maximum number of students as its capacity. Maximizing the flow through this network maximizes the number of assigned courses.

Table 1
Example of preference lists

Student	r_s	1	2	3	...	10
s040202	4	DAC03	DA247	DA125	...	DA405
s040203	4	DA619	DA125	DA201	...	DA616
s040204	1	DA418	DA242			

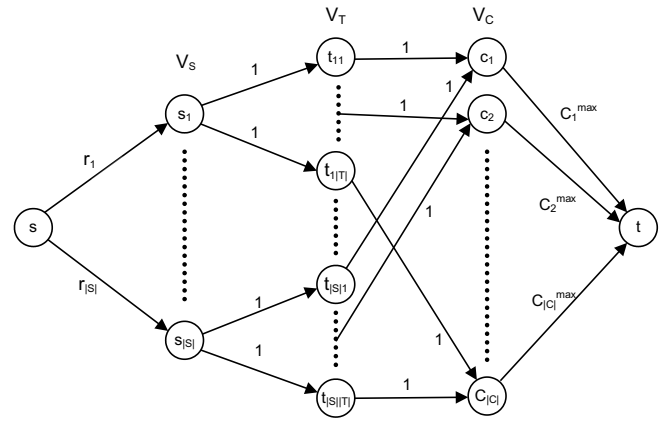

Fig. 1. The network flow model of the initial problem (formulation No. 1).

3.2. Problem formulation No. 2

As we received the first data set from the Department of Industrial Design, we were very surprised: there suddenly were also *lower bounds* $C_c^{\min}$ on the number of students participating in course c. This yields the new option of a course being canceled if there are not enough students attending. An exception to this rule is that a course cannot be canceled if it is compulsory for some students. This new option cannot be modeled within the flow model. In fact, the problem becomes NP-hard; see Section 4. After looking at the data more carefully and after conversations with the planner of the Department of Industrial Design we noticed there were additional restrictions. The remainder of this subsection explains these additional restrictions and describes the problem into more detail.

An academic year is divided into a certain number of teaching periods. For instance, the academic year 2005–2006 is divided into six teaching periods. We define such a period as a *block*. The Department of Industrial Design wants us to schedule two consecutive blocks simultaneously. Therefore, set B is introduced as the set of blocks that have to be scheduled simultaneously.

In problem formulation No. 1 we assumed the *workload* of all courses was equal. However, there are courses with a workload of 40 hours and courses with a workload of 80 hours. In the remainder of this paper a workload of 1 corresponds to a workload of 40 hours. In Section 4 we prove that having courses with a workload 1 and courses with a workload 2 makes the problem already NP-hard.

In problem formulation No. 2 the definition of r_s is adjusted into the requested total workload of student s for all blocks in B. For every student s, also a maximum requested workload r_{sb} for each block $b \in B$ separately is given. This is necessary because the requested workload of a student is not always equally divided over all blocks $b \in B$. One example is when $B = \{b_1, b_2\}$ and student s has to do a practical training in block b_2 and wants to do two courses in block b_1. In this case he sets: $r_s = 2$, $r_{sb_1} = 2$ and $r_{sb_2} = 0$.

It was assumed in problem formulation No. 1 that a course has one meeting every week, hence it has a single time slot. However, there are also courses which have more than one weekly meeting,

hence have more than one time slot. If such a course is assigned to a student, the student has to be available at all time slots.

The set C of courses contains courses with multiple sections, meaning that the course is repeated during the week. Table 2 contains course DA242 as an example. The time slots in the table are encoded. For example, the code B1TM2 stands for the second part of Tuesday morning in block 1. The course DA242 has five sections which all have two time slots.

We define I as the set of sections offered to the students. For each course there is at least one section in the set I and for every section $i \in I$ its course $c(i) \in C$ is given. In problem formulation No. 2 there are no maximum and minimum number of students for a course like in problem formulation No. 1, but a minimum number $C_i^{\min}$ and a maximum number $C_i^{\max}$ of students for each section $i \in I$. The meeting times for each section $i \in I$ are given as the set $T(i) \subseteq T$ of time slots, where T denotes the set of meeting times of the blocks in B. There are a few courses, for example literature studies, which are not assigned to a time slot and thus $T(i) = \emptyset$ for a section i of such a course.

For each section $i \in I$ and block $b \in B$ the parameter $w(i, b)$ is defined as the workload of section i in block b. Hence for a section i with a workload of 80 hours in block b we have $w(i, b) = 2$. It is necessary to define the workload for each section and each block, because there are sections with workload 2 distributed over different blocks. See for example course DA247 in Table 2. The workloads of a section in blocks 1 and 2 are denoted with *wl b*1 and *wl b*2 in Table 2. Min and Max denote the minimum and maximum number of students for a section in this table.

It is possible that courses are given in two blocks which are not scheduled simultaneously. If a student has been assigned to a section of such a course in the first block, then he needs to be assigned to the same section of this course in the second block. This implies there are students already preassigned to sections at the moment the schedule of the second block is being constructed. Therefore, we introduce the set F of *fixations* which contains combinations (s, i) for which section i is already assigned to student s.

If students have specific needs, for instance when they almost finish their studies and only have one course left to pass, a course on the preference list of the student can be set to *urgent*. Also first year students have to do some compulsory courses, these courses can also be set to urgent. Courses marked as urgent should be assigned as much as possible, even if it would lead to course attendance below the minimum. We define U as the set of combinations (s, p) for which course c_{sp} is considered urgent for student s.

In summary: the input of problem formulation No. 2 consists of:

- A set B of blocks that have to be scheduled simultaneously.
- A set T of time slots.
- A set C of courses.
- A set S of students; for every student s a total requested workload r_s, a maximum requested workload r_{sb} for each block separately, a set P_s of filled positions on the preference list and for each position $p \in P_s$ a course c_{sp} is given.
- A set I of sections; for every section i its course $c(i)$, a minimum $C_i^{\min}$ and maximum $C_i^{\max}$ number of students, a set $T(i) \subseteq T$ of time slots and its workload $w(i, b)$ for each block $b \in B$ is given.
- A set U of combinations (s, p) for which course c_{sp} is urgent for student s.
- A set F of combinations (s, i) for which section i is already preassigned to student s.

Our main goal is to assign workload to students as much as possible, while:

- Maintaining the number of students in a section below the maximum size prescribed.
- The workload assigned to student s is less than or equal to r_s.
- The workload assigned to student s in block b is less than or equal to r_{sb}.
- Sections assigned to a student do not conflict in time.
- Students are only assigned to a section of a course on their preference list.
- Students are only assigned to one section of a course.
- Student s is assigned to section i if $(s, i) \in F$.

Two soft constraints are that all urgent courses are assigned and all section size lower bounds are respected.

Table 2
Examples of courses

Course	Section	Time slots	wlb1	wlb2	Min	Max
DA242	DAG242-1	B1TM2, B1TA1	1	0	0	30
	DAG242-2	B1TM2, B1TA2	1	0	0	30
	DAG242-3	B1TM2, B1WA1	1	0	0	30
	DAG242-4	B1TM2, B1WA1	1	0	0	30
	DAG242-5	B1TM2, B1WA2	1	0	0	30
DA247	DAG247-1	B1WA2, B2WA2	1	1	5	15
	DAG247-2	B1WA2, B2WA2	1	1	5	15

4. Some complexity results

Cheng et al. (2002) prove that the Student Scheduling Problem is NP-hard. The Student Scheduling Problem is a special case of problem formulation No. 2, with as main difference that the total workload of the courses on the preference list is always equal to the requested workload. Therefore, all courses on the list of a student have to be assigned in the student scheduling problem. This proves already that the timetabling problem defined as problem formulation No. 2 is NP-hard. This does not give any guidance on the prospects of solving any particular subproblem. In the rest of this paragraph we prove again that problem formulation No. 2 is NP-hard by identifying some independent NP-hard subproblems. These subproblems result from adding one additional constraint to problem formulation No. 1.

In this section we sometimes assume that there are no time slots, which is the same as giving each course a unique time slot. In the network flow model of problem formulation No. 1 we could then connect all nodes V_S in the first layer directly to the nodes V_C of the third layer if this course is on the preference list of the corresponding student. These edges would have a capacity of one.

In the first subproblem, the additional constraint is a lower bound on the number of students in a course. There are no time slots, there is only one section for each course c with a minimum and a maximum number of participating students. The workload of all courses is one, and only one block has to be scheduled. Formally, problem $P_{\min}$ is defined as follows:

> *Instance*: A set C of courses; for every course $c \in C$ a minimum number $C_c^{\min}$ and a maximum number $C_c^{\max}$ of participating students. A set S of students; for every student $s \in S$ a preference list of some courses in C, and a number r_s of requested courses.
> *Question*: Does there exist an assignment such that (i) every student s gets exactly r_s courses from its preference list, and such that (ii) for every course c the number of assigned students is either zero (if the course does not take place) or falls between the bounds $C_c^{\min}$ and $C_c^{\max}$?

Theorem 4.1. *Problem P_{min} is NP-complete.*

Proof. The proof is done by reduction from the *exact cover by 3-sets* problem: Given a ground set $X = \{x_1, \ldots, x_n\}$ and a set $T = \{t_1, \ldots, t_m\}$ of 3-element subsets of X, can one select $T' \subseteq T$ such that every element of X occurs in exactly one member of T'? Exact cover by 3-sets is NP-complete (Garey and Johnson, 1979).

From an instance of the exact cover by 3-sets problem, we construct a corresponding instance of problem $P_{\min}$ with n students $x_1, \ldots, x_n$ and with m courses $t_1, \ldots, t_m$. Every student s has a demand of one course ($r_s = 1$), and every course c has minimum and maximum capacity three ($C_c^{\min} = C_c^{\max} = 3$).

Assume X possesses an exact cover T'. Assign student x_s to course t_c if and only if $x_s \in t_c$ and $t_c \in T'$. Since T' is an exact cover of X, every student x_s will be assigned to exactly one course t_c. The course t_c is assigned to three students if it is in T', and to zero students if it is not in T'. This shows that the constructed instance of $P_{\min}$ is a yes-instance. The converse statement can be seen in a similar way. □

In the second subproblem, we take problem formulation No. 1 and additionally allow courses with a workload of 2. We consider a situation with only one section for each course c, only a single block without any time slots and there is no minimum capacity of courses. Problem P_{wl} is defined as follows:

Instance: A set C of courses; for every course $c \in C$ a workload $wl_c \in \{1, 2\}$ and a maximum capacity $C_c^{\max}$ of participating students. A set S of students; for every student $s \in S$ a preference list of some courses in C, and a desired workload r_s.
Question: Does there exist an assignment such that (i) every student s gets courses with a total workload r_s from its preference list, and such that (ii) for every course c the number of assigned students is at most $C_c^{\max}$?

Theorem 4.2. *Problem P_{wl} is NP-complete.*

Proof. The proof is done by reduction from the 3-SAT variant where every variable occurs exactly twice in negated and exactly twice in unnegated form. Consider an arbitrary instance of this 3-SAT variant.

- For every variable x_k, we introduce two corresponding students $st(x_k)$ and $st(\overline{x_k})$ who both request a workload of two.
- For every variable x_k, we also introduce a corresponding variable-course $VC(x_k)$ which has a workload of two and a capacity of one. $VC(x_k)$ is in the preference list of $st(x_k)$ and $st(\overline{x_k})$.
- For every clause c_j, we introduce a clause-course $CC(c_j)$ with a workload of one and a capacity of two. $CC(c_j)$ is in the preference list of a student $st(x_k)$ (respectively $st(\overline{x_k})$) if and only if x_k (respectively $\overline{x_k}$) occurs as a literal in clause c_j.

Note that in any feasible assignment, student $st(x_k)$ (respectively student $st(\overline{x_k})$) will either do course $VC(x_k)$ or the two courses $CC(c_{j_1})$ and $CC(c_{j_2})$ for which literal x_k (respectively literal $\overline{x_k}$) occurs in clauses c_{j_1} and c_{j_2}.

Assume that the 3-SAT instance is a yes-instance, and consider a corresponding satisfying truth-assignment. If x_k is set to TRUE, then we assign student $st(x_k)$ to $VC(x_k)$, and student $st(\overline{x}_k)$ to the two clause-courses that correspond to the clauses containing $\overline{x}_k$. If x_k is set to FALSE, we assign $st(x_k)$ to the clause-courses that correspond to the clauses containing x_k, and student $st(\overline{x_k})$ to $VC(x_k)$. Then each student receives his requested workload, and every course $VC(x_k)$ gets only a single student. Since every clause has at most two FALSE literals, the corresponding clause-course will get at most two students. So every yes-instance of the 3-SAT problem leads to a yes-instance of the timetabling problem.

Now assume that the constructed instance of problem P_{wl} is a yes-instance. Then every student $st(x_k)$ receives a workload of 2, which implies that the student must either be assigned to a variable-course $VC(x_k)$, or to two clause-courses $CC(c_{j_1})$ and $CC(c_{j_2})$. If student $st(x_k)$ is assigned to $VC(x_k)$, we set x_k to TRUE. If student x_k is assigned to two clause-courses, then we set x_k to FALSE. Since each clause-course $CC(c_j)$ is assigned to at most two students, every clause contains at most two FALSE literals. Hence, every yes-instance of P_{wl} corresponds to a yes-instance of 3-SAT. □

In the third subproblem the possibility that a course has multiple time slots is added to problem formulation No. 1. We consider the timetabling problem where all courses have a workload of one, there is no minimum capacity for courses, one section for each course c and one block has to be scheduled. Problem P_{mt} is defined as follows:

Instance: A set T of time slots; A set C of courses; for every course $c \in C$ a set $T(c) \subseteq T$ of time slots and a maximum capacity $C_c^{\max}$ of participating students. A set S of students; for every student $s \in S$ a preference list of some courses in C, and a number of requested courses r_s.
Question: Does there exist an assignment such that (i) every student s gets exactly r_s courses from its preference list, and (ii) for every course c the number of assigned students is at most $C_c^{\max}$, and such that (iii) the assigned courses to a student do not have a time slot in common?

Theorem 4.3. *Problem P_{mt} is NP-complete.*

Proof. The proof is done by reduction from the *independent set* problem: Given a graph $G = (V, E)$ and an integer k, does there exist a subset $V' \subseteq V$ with $|V'| \geqslant k$ and $|V| = n$ such that no two vertices in V' are connected?

From an arbitrary instance of the independent set problem we construct a corresponding instance of problem P_{mt} with one student that requests k courses. For every edge $e \in E$ we introduce a time slot t_e and for every node $v \in V$ we introduce a course c_v with capacity one and time slots $t_e \in T(c_v)$ if v is a vertex of edge e. Note that the number of time slots of a course is equal to the degree of the corresponding vertex in G. The preference list of the student contains all n courses.

Assume that the independent set instance is a yes-instance with independent set V' and $|V'| = k$. If $v \in V'$, then c_v is assigned to the student. The student receives its requested k courses and the assigned courses have no time slot in common. Trivially, all courses have no more students assigned than their capacity.

Assume that the constructed instance of problem P_{mt} is a yes-instance. Then the student is assigned to k courses which all have different time slots. This implies that the nodes corresponding with an assigned course are not connected and that the selected nodes form an independent set. Hence, every yes-instance of P_{mt} corresponds to a yes-instance of independent set. □

The independent set problem is polynomial solvable in the case all nodes have degree less than or equal to two. Therefore, the special case of problem P_{mt} where every course has one or two time slots is not proven to be NP-complete. The complexity result of this problem is given in the following theorem.

Theorem 4.4. *P_{mt} is NP-complete, even if restricted to the class of instances in which $|T(c)| \leqslant 2$, $\forall c \in C$.*

Proof. The proof is done by reduction from the *chromatic index* problem: Given a simple graph $G = (V, E)$ and a positive integer K, can E be partitioned into disjoint sets $E_1, \ldots, E_k$, with $k \leqslant K$, such that, for $1 \leqslant i \leqslant k$, no two edges in E_i share a common endpoint in G.

From an instance of the chromatic index problem, we construct a corresponding instance of problem P_{mt} where each course has

one or two time slots. For every node $v_i \in V$ we introduce a time slot $t(v_i)$ and for every edge $e_j \in E$ we introduce a course $C(e_j)$ with a capacity of 1 student. If $e_j = \{v_{i_1}, v_{i_2}\}$, then course $C(e_j)$ has time slots $t(v_{i_1})$ and $t(v_{i_2})$. We introduce K students, $s_1, \ldots, s_K$, which all have a preference list containing all courses. We define $p \equiv |E|(\mathrm{mod}K)$. Students s_1 to s_p request $\lceil |E|/K \rceil$ courses and students s_{p+1} to s_K request $\lfloor |E|/K \rfloor$ courses.

Assume G contains a chromatic index K. Consider K sets E_i in the partition, of which some may be empty. Let E_1 and E_2 be two disjoint sets with $|E_1| > |E_2| + 1$, then there exist two disjoint sets E'_1 and E'_2 such that $|E'_1 \cup E'_2| = |E_1 \cup E_2|$ and $|E'_1| = |E_1| - 1$ and $|E'_2| = |E_2| + 1$ and both E'_1 and E'_2 form a matching. Therefore we can assume: $||E_i| - |E_j|| \leqslant 1 \forall i, j$. This implies that there exists a solution to the chromatic index problem with p sets containing $\lceil |E|/K \rceil$ edges and the other sets containing $\lfloor |E|/K \rfloor$ edges. Now assign student s_i to course $C(e_j)$ if $e_j \in E_i$. The first p students are assigned to the courses corresponding to the p bigger sets E_i, so each student gets its requested number of courses. The courses assigned to a student all have different time slots, because the edges in this set form a matching. Each edge is assigned to at most one set E_i, therefore each course is also assigned to at most one student. So every yes-instance of the chromatic index problem leads to a yes-instance of the timetabling problem.

Now assume that the constructed instance of problem P_{mt} is a yes-instance. If student s_i is assigned to course $C(e_j)$ then add edge e_j to set E_i. All assigned courses have different time slots, therefore the edges in the same set are not connected and form a matching. There are K students which result in K disjoint sets E_i and all $|E|$ courses have been assigned, hence every constructed yes-instance of P_{mt} corresponds to a yes-instance of the chromatic index problem. □

The fourth subproblem considers problem formulation No. 1, but courses with multiple sections are added. We define this problem as P_{ms}. In this problem every course has a workload of one, no minimum number of students, only one block has to be scheduled and each section has at most one time slot. Problem P_{ms} is defined as follows:

Instance: A set T of time slots; A set C of courses; A set I of sections; for every section $i \in I$ its course $c(i)$, a time slot $t_i \in T$ and a maximum capacity $C_i^{\max}$ of participating students. A set S of students; for every student $s \in S$ a preference list of some courses in C, and a number r_s of requested courses.
Question: Does there exist an assignment such that (i) every student s is assigned to exactly r_s sections of courses from its preference list, and (ii) for every section i the number of assigned students is at most $C_i^{\max}$ and (iii) a student is assigned to only one section of a course and (iv) the assigned sections to a student do not have a time slot in common?

Theorem 4.5. *Problem P_{ms} is NP-complete.*

Proof. The proof is done by reduction from the 3-SAT problem. Consider an arbitrary instance of the 3-SAT problem.

- For each variable x in clause c_j we introduce a variable-course $VC(c_j,x)$. These courses have two literal-sections, $LS(c_j,x)$ and $LS(c_j,\bar{x})$, with capacity one. The time slots are chosen such that $T(LS(c_j,\bar{x})) = T(LS(c_{j+1},x))$ and $T(LS(c_n,\bar{x})) = T(LS(c_1,x))$ where n is the number of clauses in which variable x occurs.
- For every clause c we introduce a clause-course $CC(c)$ with one clause section $CS(c)$ with capacity 2 and time slot $T(CS(c))$.
- For every literal l in clause c we introduce a dummy-course $DC(c, l)$. For each dummy-course $DC(c, l)$ we introduce one dummy-section $DS(c, l)$, with capacity 1. If l is the literal corresponding with variable x, then time slot of this dummy-section is equal to $T(LS(c, l))$.
- For every variable x, we introduce one student $st(x)$ that requests a number of courses equal to the number of times the variable x occurs in a clause. Its preference list contains the variable-courses $VC(c, x)$ if clause c contains variable x.
- For every literal l in clause c we introduce a student $st(c, l)$ that requests two courses from its preference list that contains three courses. Assume l is a literal of the variable x, then student $st(c, l)$ requests variable-course $VC(c, x)$, clause-course $CC(c)$ and the dummy-course $DC(c, l)$.

Assume that the 3-SAT instance is a yes-instance, and consider a corresponding truth-assignment. If variable x is TRUE, then $st(x)$ is assigned to the literal-sections $LS(c, x)$. If literal l of clause c is TRUE, then assign student $st(c, l)$ to literal section $LS(c, \bar{l})$ and $DS(c, l)$. For each FALSE literal l, student $st(c, l)$ is assigned to $LS(c, l)$ and to the clause-section $CS(c)$. All the sections assigned to $st(c, l)$ have different time slots and the requested number of courses for student $st(c, l)$ is assigned. Because clause c is TRUE, at most two students are assigned to $CS(c)$ and all other sections are only assigned to at most one student. So every yes-instance of the 3-SAT problem leads to a yes-instance of the timetabling problem.

Now assume that the constructed instance of the timetabling problem is a yes-instance. At most two students are assigned to $CC(c)$. Therefore, at least one of the literals of the clause c is not assigned and evaluates to TRUE. If student $st(x)$ is assigned to the literal-sections $LS(c, x)$, then the variable x is evaluated to TRUE and if this student is assigned to the $LS(c, \bar{x})$ it is FALSE. If student $st(c, l)$ is assigned to literal section $LS(C, \bar{l})$, then the literal l is evaluated to TRUE. So every yes-instance of the timetabling problem leads to a yes-instance of the 3-SAT problem. □

Now we consider the special case of problem P_{ms}, where there are no time slots. The complexity result of this problem is given in the following theorem.

Theorem 4.6. *Problem P_{ms} restricted to the class of instances without time slots is polynomially solvable.*

Proof. The problem can be modeled as a network flow model. An illustration of the network with its arc capacities can be found in Fig. 2.

The layered network has a source node connected to the first layer of nodes of which each node corresponds to a student. The set of student nodes is denoted with V_S. The capacity of the arc between a student node and the source node is equal to the requested number of courses of the student and takes care that a student does not get more courses assigned than requested.

For each combination of a student and a course a node is defined in the second layer. This layer is denoted as the set V_C of course-nodes which contains $|S||C|$ nodes. There is an outgoing arc with capacity one from a student node to a node in V_C if the course is on the preference list of the corresponding student. Each student node has therefore a number of outgoing arcs equal to the length of the preference list. In Fig. 2 both student 1 and student $|S|$ have course 1 and course $|C|$ on their preference lists.

The third layer of the graph contains a node for every section. This set of section-nodes is defined as V_I. Between section-nodes and course-nodes there is an arc with capacity 1 if this section belongs to this course. For example, in Fig. 2 course number 1 has two sections (I_1 and I_2) and course $|C|$ has one section ($I_{|I|}$).

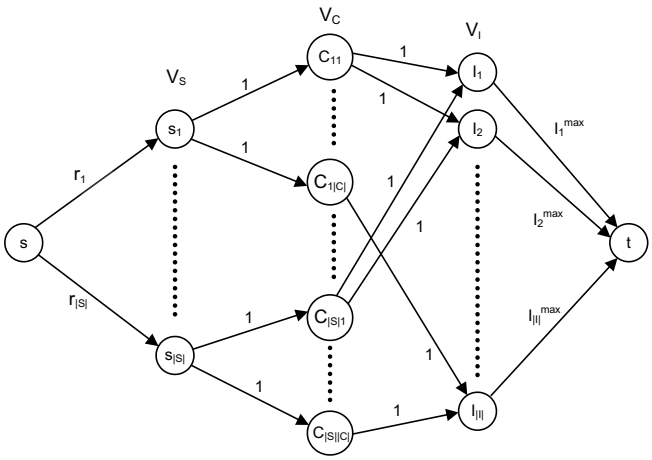

Fig. 2. The network flow model for problem P_s without time slots.

Each section-node is connected to the sink with its maximum number of students as its capacity. Maximizing the flow through this network maximizes the number of assigned courses. □

5. The integer linear programming model

To build a schedule which best fits the needs for the students, the problem is formulated and solved as a *lexicographic optimization* problem. Lexicographic optimization is a form of multi-criteria optimization. Assume we have objectives f_i, $i = 1, \ldots, m$. If solution x_1 optimizes f_1, then we say solution x_2 optimizes f_2 under the condition $f_1(x_1) = f_1(x_2)$. Recursively x_i optimizes f_i under the constraining conditions that $f_1(x_i) = f_1(x_1), \ldots, f_{i-1}(x_i) = f_{i-1}(x_{i-1})$.

The timetabling problem is split into four subproblems which are formulated as an integer linear programming problem. The goals of the four subproblems are:

(1) Maximize the number of assigned courses with an urgency.
(2) Minimize the shortage of students to reach the minimum number of students of a section. Because of urgencies, some sections must be taught, but may not have enough students with the corresponding course on their preference list. As many students as possible are assigned to those sections.
(3) Maximize the total assigned workload. We try to assign a workload r_s to every student s.
(4)
   - Minimize the average position of the assigned courses.
   - Minimize the number of students that are assigned to more than one course on the positions 7 up to 10 on their preference list.
   - Minimize the spread of the students over the sections.

All parameters have already been introduced in Section 3. What is left to define is the decision variables. These are defined as follows:

$$x_{sp} := \begin{cases} 1 & \text{if course } c_{sp} \text{ is assigned to student } s, \\ 0 & \text{otherwise}, \end{cases}$$

$$y_i := \begin{cases} 1 & \text{if section } i \text{ is assigned to one or more students}, \\ 0 & \text{otherwise}, \end{cases}$$

$$z_{si} := \begin{cases} 1 & \text{if section } i \text{ is assigned to student } s, \\ 0 & \text{otherwise}. \end{cases}$$

The following constraints have to be fulfilled in all four subproblems:

$$\sum\nolimits_{i \in I | c_{sp} = c(i)} z_{si} = x_{sp} \quad \forall s \in S, \ \forall p \in P_s, \tag{1}$$

$$\sum\nolimits_{p \in P_s} \sum_{i \in I | c_{sp} = c(i)} w(i,b) z_{si} \leqslant r_{sb} \quad \forall s \in S, \ \forall b \in B, \tag{2}$$

$$\sum\nolimits_{p \in P_s} \sum_{i \in I | c_{sp} = c(i)} \sum_{b \in B} w(i,b) z_{si} \leqslant r_s \quad \forall s \in S, \tag{3}$$

$$\sum\nolimits_{s \in S} \sum_{p \in P_s | c_{sp} = c(i)} z_{si} \leqslant C_i^{\max} y_i \quad \forall i \in I, \tag{4}$$

$$\sum\nolimits_{p \in P_s} \sum_{i \in I | c_{sp} = c(i), t \in T(i)} z_{si} \leqslant 1 \quad \forall s \in S, \ \forall t \in T, \tag{5}$$

$$z_{si} = 1 \quad \forall s \in S, \ \forall i \in I | (s,i) \in F, \tag{6}$$

$$x_{sp} \in \{0,1\} \quad \forall s \in S, \ \forall p \in P_s, \tag{7}$$

$$y_i \in \{0,1\} \quad \forall i \in I, \tag{8}$$

$$z_{si} \in \{0,1\} \quad \forall s \in S, \ \forall i \in I. \tag{9}$$

Constraint (1) takes care that at most one section of a course is assigned to a student. The workload assigned to a student has to be less than or equal to the requested workload each block separately and all blocks together. This is fulfilled by constraints (2) and (3). Constraint (4) enforces that the maximum number of students for a section is not exceeded and constraint (5) imposes that at each time slot only one section is assigned to each student. If $(s, i) \in F$ then section i has to be assigned to student s, which is fulfilled by constraint (6).

As explained above, the problem is split into four subproblems which are solved sequentially. The goal of the first subproblem is to maximize the number of assigned courses with an urgency. The constraint that a section needs to have more than a minimum number of students is not a restriction in this subproblem, because at least one section of a course must be given if there is a student with an urgency for this course. This first subproblem can be solved with the following ILP formulation:

$$\begin{aligned} \max \quad & \sum_{(s,p) \in U} x_{sp} = U^{\max} \\ \text{subject to}: \quad & (x, y, z) \text{ satisfy } (1)-(9). \end{aligned}$$

The second subproblem minimizes the total shortage $S_{\min}$ of students to reach the minimum number of students of a section, keeping the maximum number of assigned courses with an urgency equal to $U^{\max}$. Sections that have to be given because they are assigned to students with an urgency for the corresponding course, are assigned to other students such that the minimum number of students for those sections is reached. The decision variable s_i is defined as the shortage of students for section i and gets a value larger than zero if it is not possible to assign section i to its minimum number $C_i^{\min}$ of students. This gives the following ILP formulation:

$$\begin{aligned} \min \quad & \sum_{i \in I} s_i = S^{\min} \\ \text{subject to}: \quad & \sum\nolimits_{(s,p) \in U} x_{sp} = U^{\max}, \\ & \sum\nolimits_{s \in S} \sum_{p \in P_s | c_{sp} = c(i)} z_{si} + s_i \geqslant C_i^{\min} y_i \quad \forall i \in I, \\ & s_i \in \mathbb{Z}_+ \quad \forall i \in I, \\ & (x, y, z) \text{ satisfy } (1)-(9). \end{aligned}$$

The third subproblem maximizes the total workload assigned to students with the restrictions that $U^{\max}$ and $S^{\min}$ keep their optimal values. This maximum workload is denoted by $W^{\max}$ and is determined by the following model:

$$\begin{aligned}
\max \quad & \sum_{s\in S}\sum_{p\in P_s}\sum_{i\in I|c_{sp}=c(i)}\sum_{b\in B} w(i,b)z_{si} = W^{\max} \\
\text{subject to}: \quad & \sum_{i\in I} s_i = S^{\min}, \\
& \sum_{(s,p)\in U} x_{sp} = U^{\max}, \\
& \sum_{s\in S}\sum_{p\in P_s|c_{sp}=c(i)} z_{si} + s_i \geqslant C_i^{\min} y_i \quad \forall i\in I, \\
& s_i \in \mathbb{Z}_+ \quad \forall i \in I, \\
(x,y,z) \text{ satisfy } & (1)-(9).
\end{aligned}$$

To 'optimize' the timetable a fourth subproblem is solved. In this subproblem we try to assign as much courses on top of the preference lists, to spread students as equally as possible over the sections of a course and to discourage that one student gets a lot of courses which are on the bottom of his preference list. The objective function is separated into three terms and has to be minimized under the restrictions that $U^{\max}$, $S^{\min}$ and $W^{\max}$ keep their optimal values.

The first term in the objective function of the fourth subproblem is:

$W_p\sum_{s\in S}\sum_{p\in P_s}\sum_{i\in I|c_{sp}=c(i)}\sum_{b\in B} w(i,b)(82-(10-p)^2)x_{sp}$. It assigns courses as high as possible on the preference lists. Note that it contains a weighing factor W_p. We chose the value 82, so that assigning a course on top of a preference list, $p = 1$, adds only once the workload to the objective function. If a course on the bottom of the list, $p = 10$, is assigned, this adds 82 times the workload to the objective function.

If a course has multiple sections, students have to be spread as equally as possible over the sections. Therefore, $I_c^{\max}$ is introduced as the number of students assigned to the section of course c with the most students assigned. Also the spread S_c of course c is introduced and is equal to the sum over all sections of course c of the difference between $I_c^{\max}$ and the assigned number of students in each section. S_c is added to the objective function with a weighing factor W_s.

We also discourage that one student gets a lot of courses of his 7th up to 10th position of his preference list. If a student gets more than one course from these positions, then a penalty W_e is paid for each 'extra' course from these positions. Therefore, the decision variable E_s is introduced for every student s. This variable is equal to the 'extra' number of courses assigned to student s which are on the 7th up to 10th position of his preference list.

This results into the final ILP formulation:

$$\begin{aligned}
\min \quad & W_p\sum_{s\in S}\sum_{p\in P_s}\sum_{i\in I|c_{sp}=c(i)}\sum_{b\in B} w(i,b)(82-(10-p)^2)z_{si} \\
& + W_s\sum_{c\in C} S_c + W_e\sum_{s\in S} E_s \\
\text{subject to}: \quad & \sum_{s\in S}\sum_{p\in P_s|c_{sp}=c} z_{si} \leqslant I_c^{\max} \quad \forall i\in I,\ \forall c\in C|c(i)=c, \\
& \sum_{i\in I|c=c(i)}\Big(I_c^{\max} - \sum_{s\in S}\sum_{p\in P_s|c_{sp}=c} z_{si}\Big) = S_c \quad \forall c\in C, \\
& \sum_{p=7}^{10} x_{sp} \leqslant 1 + E_s \quad \forall s\in S, \\
& \sum_{s\in S}\sum_{p\in P_s}\sum_{i\in I|c_{sp}=c(i)}\sum_{b\in B} w(i,b)z_{si} = W^{\max}, \\
& \sum_{i\in I} s_i = S^{\min}, \\
& \sum_{(s,p)\in U} x_{sp} = U^{\max}, \\
& \sum_{s\in S}\sum_{p\in P_s|c_{sp}=c(i)} z_{si} + s_i \geqslant C_i^{\min} y_i \quad \forall i\in I, \\
& E_s \in \mathbb{Z}_+ \quad \forall s\in S, \\
& I_c^{\max}, S_c \in \mathbb{Z}_+ \quad \forall c\in C, \\
& s_i \in \mathbb{Z}_+ \quad \forall i\in I, \\
(x,y,z) \text{ satisfy } & (1)-(9).
\end{aligned}$$

The first constraint enforces that $I_c^{\max}$ will contain the maximum number of students assigned to one of the sections of course c. The spread of the students over the sections of course c is determined by the second constraint. The value of E_s becomes positive if more than one course on the positions 7–10 of the preference list of student s is assigned. The third constraint ensures this. All other constraints have been explained before.

6. The computational results

The computational results for the academic year 2005–2006 are given in this section. This academic year was divided into six blocks. Blocks 1 & 2, blocks 3 & 4 and blocks 5 & 6 are scheduled simultaneously. In all blocks the meetings were on Tuesday morning, Tuesday afternoon, Wednesday morning and Wednesday afternoon. Every morning and afternoon is split into two parts. So all blocks contain eight time slots. More details about the input are given in Table 3. The abbreviation wl stands for workload.

The number of students requesting workload decreased during the academic year, because of students who were doing a practical training and students who were finishing their studies. In blocks 5 & 6 the average requested workload per student is larger than in blocks 3 & 4, which is caused by students who still hope to reach the required workload for the academic year by doing some 'extra' courses.

The large number of urgencies in blocks 1 & 2 can be explained by the fact that first-year students are preassigned to courses, because they are not able to make a choice themselves. All first-year students have six compulsory courses, which they have to do in the first-year. These six courses are set as urgent and only four out of these six courses can be done in each block. The urgencies in blocks 5 & 6 can be explained by students who are finishing their education and only have some specific courses left to do.

The models introduced in Section 5 are solved by the standard IP solver CPLEX 10.0. The computations are done on an Intel Pentium M, 2.0 GHz processor with 1.0 GB internal memory. The values of the weighing factors were $W_p = 10$, $W_s = 1$ and $W_e = 100$. Assigning courses on top of the preference list is much more important than trying to spread the students over sections. Therefore, W_s is chosen much smaller. To take care that not one specific student gets only courses on the bottom of his list, we choose a value of W_e that is significantly bigger than W_p. The results for the academic year 2005–2006 are given in Table 4.

The computation time of CPLEX given in Table 4 is the computation time of CPLEX for solving the fourth subproblem. The computation time of the first three subproblems is even shorter.

Table 3
Input information for academic year 2005–2006

Blocks	$\|T\|$	$\|S\|$	$\|C\|$	$\|I\|$	$\|U\|$	Available wl	Requested wl
1 & 2	16	356	51	79	590	1504	1416
3 & 4	16	328	64	88	279	1545	1288
5 & 6	16	302	58	89	151	1544	1333

Table 4
Results for the academic year 2005–2006

	Block 1 & 2	Block 3 & 4	Block 5 & 6
Runtime CPLEX (s)	1.38	1.53	1.67
$U^{\max}$	439	273	134
$S^{\min}$	0	0	0
$W^{\max}$	1369	1261	1300
Average position	3.30	3.64	3.87
Bad positions	8	17	39

What can be concluded is that the computation time of CPLEX is negligible.

In blocks 1 & 2 a requested workload of 1416–1369 = 47 could not be assigned, in blocks 3 & 4 a requested workload of 27 and in blocks 5 & 6 a requested workload of 33 could not be assigned. Especially in blocks 1 & 2 this is partly caused by the small difference between the requested and available workload. However, the main causes are preference lists for which it was impossible to assign the requested workload. Examples of such wrongly chosen preference lists are:

- An empty preference list, because students did not hand it in on time.
- A preference list with less than 10 courses.
- A preference list with not enough different time slots in one of the two blocks.
- A preference list with the same course in more positions. There was even a student with ten times the same course on his preference list.

If all students would hand in a preference list with 10 different courses and enough different time slots, then in blocks 1 & 2 only five students would not be assigned to their requested number of courses and in blocks 3 & 4 and blocks 5 & 6 only three students.

Table 4 also shows that in blocks 1 & 2 only 439 out of 590 urgency requests could be assigned. This can be explained by the fact that in these blocks all six courses on the preference list of first-year students are set as urgent, of which at most 4 are assigned. This gives at least two not assigned courses with an urgency for each first-year student.

The average position denotes the average of the positions of the courses assigned to a student. On average students request a workload of 4, which usually corresponds to four courses. For example, if courses on the positions 1, 3, 5 and 7 are assigned, then the average position for this student is 4. Hence, it can be concluded that students are assigned a lot of courses which are at the top of their preference list.

During the academic year, the average position increases. This can be explained by the fact that in blocks 5 & 6 the first-year students are allowed to choose four courses from the same course pool as the other students. In blocks 3 & 4 they were usually assigned to two out of the six obligatory courses and to two courses chosen by themselves. In blocks 1 & 2 they were preassigned to obligatory courses.

If a student is assigned to $i \geqslant 1$ courses on 7th up to 10th position on his preference list he has $i - 1$ bad positions. Also from the number of bad positions it can be concluded that students are assigned to courses at the top of their preference lists. The number of bad positions increasing during the year, is again due to the first-year students.

7. Conclusions

We have formulated, analyzed and solved a real-world timetabling problem that showed up at the Department of Industrial Design of the TU Eindhoven. Our successful approach was based on an Integer Linear Programming formulation. The running time that CPLEX needs for solving the resulting instances is negligible. An advantage of an ILP approach is its flexibility. Our experience is that the constraints of the timetabling problem change every academic year and even during the academic year.

The administration and the students of the Department of Industrial Design were highly satisfied with the timetables generated by our program. Most students now receive courses that are at the top of their preference lists. There still are a few students who are not satisfied, but in most cases this turns out to be solely their own fault: they failed to specify correct preferences in the correct format.

References

Ahuja, R.K., Magnanti, T.L., Orlin, J.B., 1993. Network Flows: Theory, Algorithms and Applications. Prentice Hall, Englewood Cliffs, New Jersey.

Avella, P., Vasilev, I., 2005. A computational study of a cutting plane algorithm for university course timetabling. Journal of Scheduling 8, 497–514.

Burke, E.K., Kingston, J., Jackson, K., Weare, R., 1997. Automated university timetabling: The state of the art. The Computer Journal 40 (9), 565–571.

Burke, E.K., Petrovic, S., 2002. Recent research directions in automated timetabling. European Journal of Operations Research 140 (2), 266–280.

Busam, V.A., 1967. An algorithm for class scheduling with section preference. Communications of the ACM 10 (9), 567–569.

Carter, M.W., Laporte, G., 1995. Recent developments in practical examination timetabling. In: Practice and Theory of Automated Timetabling I, Burke and Ross. Lecture Notes in Computer Science, vol. 1153. Springer-Verlag.

Carter, M.W., Laporte, G., 1998. Recent developments in practical course timetabling. In: Practice and Theory of Automated Timetabling II, Burke and Carter. Lecture Notes in Computer Science, vol. 1408. Springer-Verlag.

Cheng, E., Kruk, S., Lipman, M., 2002. Flow formulations for the student scheduling problem. In: Practice and Theory of Automated Timetabling IV, Burke and De Causmaecker. Lecture Notes in Computer Science, vol. 2740. Springer-Verlag.

Daskalaki, S., Birbas, T., 2005. Efficient solutions for a university timetabling problem through integer programming. European Journal of Operations Research 160 (1), 106–120.

de Werra, D., 1985. An introduction to timetabling. European Journal of Operations Research 19, 151–162.

Dimopoulou, M., Miliotis, P., 2001. Implementation of a university course and examination timetabling system. European Journal of Operations Research 130 (1), 202–213.

Even, M., Itai, A., Shamir, A., 1976. On the complexity of timetable and multicommodity flow problems. SIAM Journal of Computing 5, 691–703.

Feldman, R., Golumbic M.C., 1989. Constraint satisfiability algorithms for interactive student scheduling. In: Proceedings of the 11th International Joint Conference on Artificial Intelligence (IJCAI 89), pp. 1010–1016.

Garey, M.R., Johnson, D.S., 1979. Computers and Intractability – A Guide to NP-completeness. W.H. Freeman and Company, San Francisco.

Laporte, G., Desrochers, S., 1986. The problem of assigning students to course sections in a large engineering school. Computational Operations Research 13, 387–394.

Lewis, R., Paechter, B., 2007. Finding feasible timetables using group-based operators. IEEE Transactions on Evolutionary Computation 11 (3), 397–413.

Miyaji, I., Ohno, K., Mine, H., 1981. Solution method for partitioning students into groups. European Journal of Operations Research 33, 82–90.

Petrovic, S., Burke, E.K., 2004. University timetabling. Handbook of Scheduling: Algorithms, Models, and Performance Analysis. CRC Press (Chapter 45).

Sabin, G.C.W., Winter, G.K., 1986. The impact of automated timetabling on universities-a case study. Journal of Operations Research Society 37, 689–693.

Schaerf, A., 1999. A survey of automated timetabling. Artificial Intelligence Review 13 (2), 87–127.

Schmidt, G., Ströhlein, T., 1980. Timetable construction – An annotated bibliography. The Computer Journal 23, 307–316.

Tripathy, A., 1992. Computerised decision aid for timetabling – A case analysis. Discrete Applied Mathematics 35 (3), 313–323.